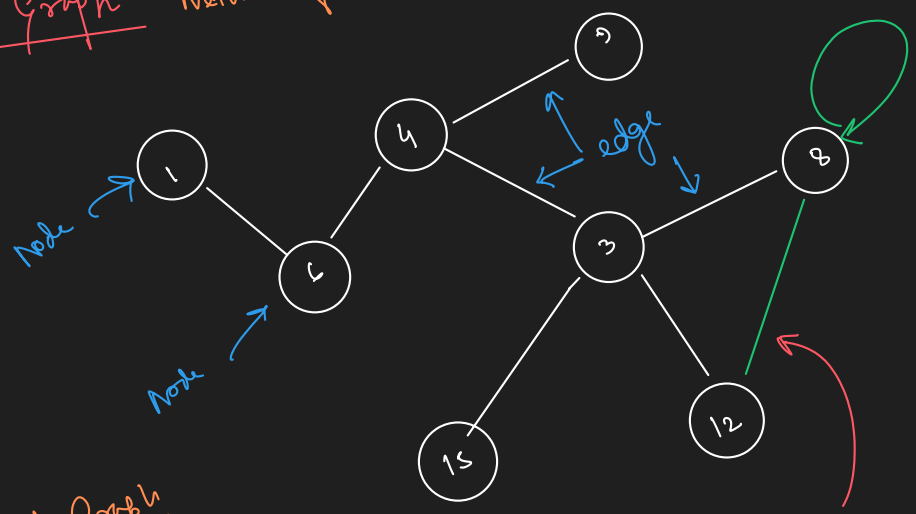
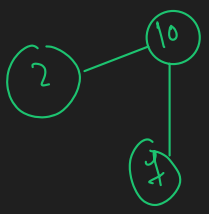
day-47

Graph Network of Nodes





Unweighted Graph

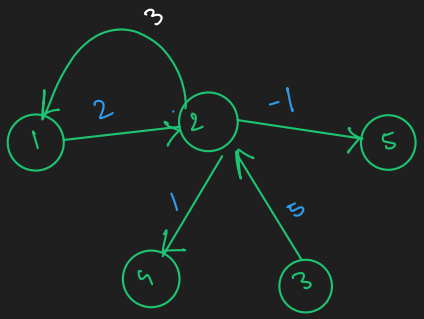
Directed Graph

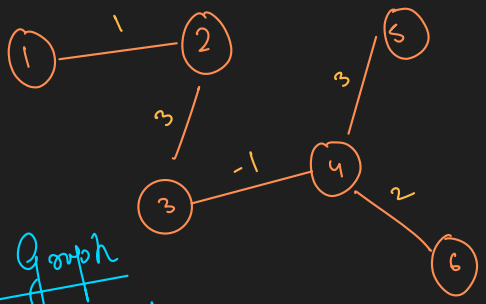
Undirected Graph

Uni-directed

bi-directed

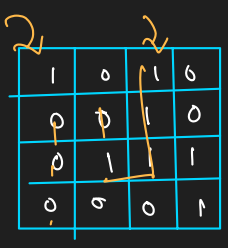
Weighted Graph



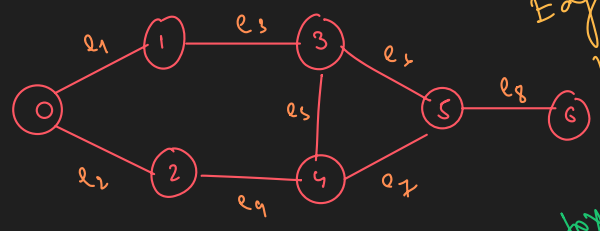


Storing a Graph

- Adjacency List
- Adjacency Matrix
- Edge List ← input
- 2D Mtx [implicit Graph] ← part of Question



day-48



Edge List → [(0,1), (0,2), (1,3), (2,4), (3,4), (4,5), (3,5), (5,6)]

Space = O(e)

Adj. List

0 → { e1 e2 }

1 → { e1, e3 }

2 → { e2, e4 }

3 → { e3, e5, e6 }

4 → { e4, e5, e7 }

5 → { e6, e7, e8 }

6 → { e8 }

vertex → Array [ArrayList] ← edge

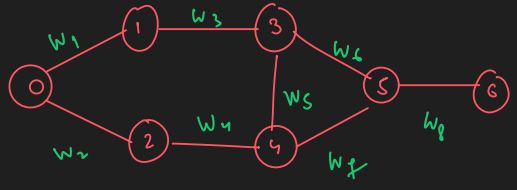
size = no. of vertex (V)

Space = O(V + 2e)

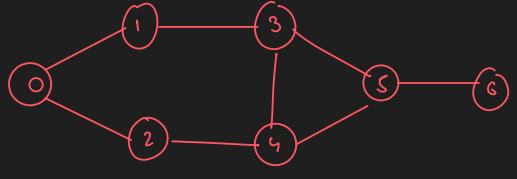
Adj Mtx

	0	1	2	3	4	5	6
0	0	w_1	w_2	0	0	0	0
1	w_1	0	0	w_3	0	0	0
2	w_2	0	0	0	w_4	0	0
3	0	w_3	0	0	w_5	w_6	0
4	0	0	w_4	w_5	0	w_7	0
5	0	0	0	w_6	w_7	0	w_8
6	0	0	0	0	0	w_8	0

Space = $O(V^2)$ ↑



Adj. List



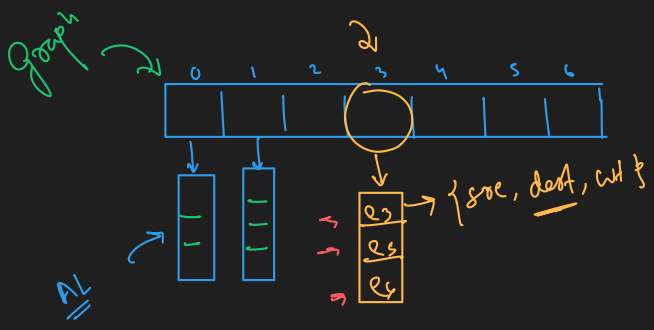
V = 7 = graph.length

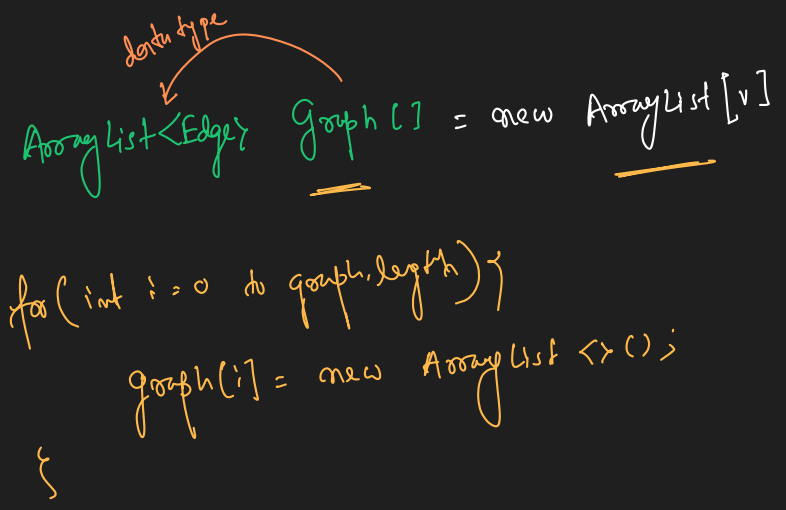
class Edge (src, dest, wt) {

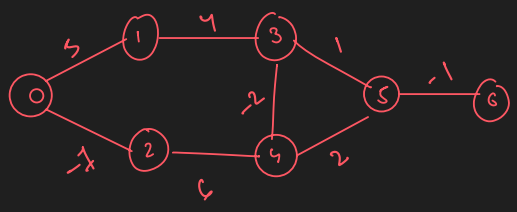
=

}

Graph → Array of AL =








```java
static class Edge{
    int src;
    int dest;
    int wt;

    public Edge(int src, int dest, int wt){
        this.src = src;
        this.dest = dest;
        this.wt = wt;
    }
}
```

```java
public static void createGraph(ArrayList<Edge>[] graph){
    for(int i=0; i<graph.length; i++){
        graph[i] = new ArrayList<>();
    }

    graph[0].add(new Edge(src: 1, dest: 0, wt: 5));
    graph[0].add(new Edge(src: 0, dest: 2, -7));

    graph[1].add(new Edge(src: 0, dest: 1, wt: 5));
    graph[1].add(new Edge(src: 1, dest: 3, wt: 4));

    graph[2].add(new Edge(src: 2, dest: 0, -7));
    graph[2].add(new Edge(src: 2, dest: 4, wt: 6));

    graph[3].add(new Edge(src: 3, dest: 1, wt: 4));
    graph[3].add(new Edge(src: 3, dest: 4, -2));
    graph[3].add(new Edge(src: 3, dest: 5, wt: 1));


    graph[4].add(new Edge(src: 4, dest: 2, wt: 6));
    graph[4].add(new Edge(src: 4, dest: 3, -2));
    graph[4].add(new Edge(src: 4, dest: 5, wt: 2));


    graph[5].add(new Edge(src: 5, dest: 3, wt: 1));
    graph[5].add(new Edge(src: 5, dest: 4, wt: 2));
    graph[5].add(new Edge(src: 5, dest: 6, -1));


    graph[6].add(new Edge(src: 6, dest: 5, -1));
}
```

Q Find Nearest Neighbour . (3)

```java
public static void nearestNeighbour(ArrayList<Edge>[] graph, int ver){

    for(int i=0; i<graph[ver].size(); i++){
        Edge curr = graph[ver].get(i);
        System.out.println(curr.dest);
    }
}
```

```java
public static void main(String[] args) {
    int V = 7;
    @SuppressWarnings("unchecked")
    ArrayList<Edge> graph[] = new ArrayList[V];

    createGraph(graph);

    // Nearest Neighbour
    int ver = 6;
    nearestNeighbour(graph, ver);

}
```

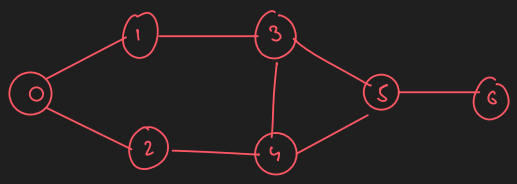
Day - 99

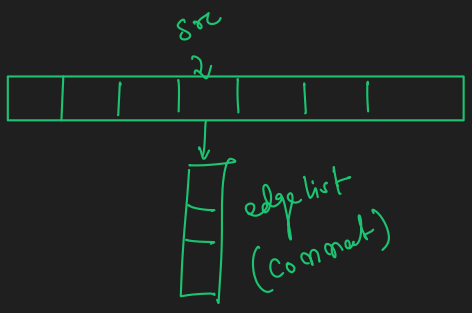
i/p 2

Edge List → [(0,1), (0,2), (1,3), (2,4), (3,4), (4,5), (3,5), (5,6)]

edge → (Soc, dest)

[Undirected] Graph → Adj. List



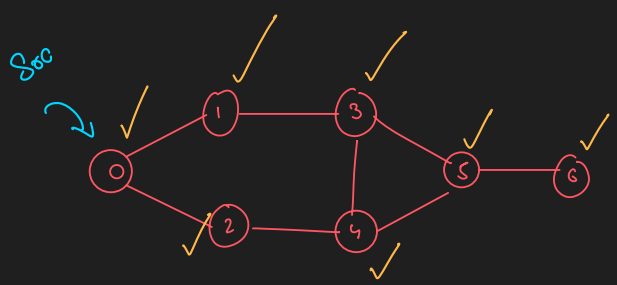


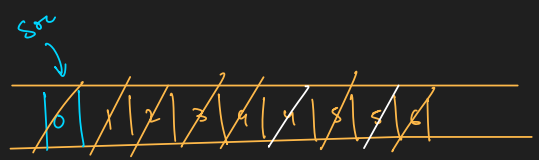

```java
public static void createGraph(ArrayList<Edge>[] graph, int[][] edgeList){
    for(int i=0; i<graph.length; i++){
        graph[i] = new ArrayList<>();
    }

    for(int i=0; i<edgeList.length; i++){
        int src = edgeList[i][0];
        int dest = edgeList[i][1];

        graph[src].add(new Edge(src, dest));
        graph[dest].add(new Edge(dest, src));
    }
}
```

Graph Traversal

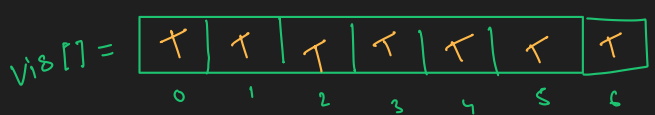
1. BFS [Breadth First Search]
2. DFS [Depth First Search]

Connected Component & Undirected Graph



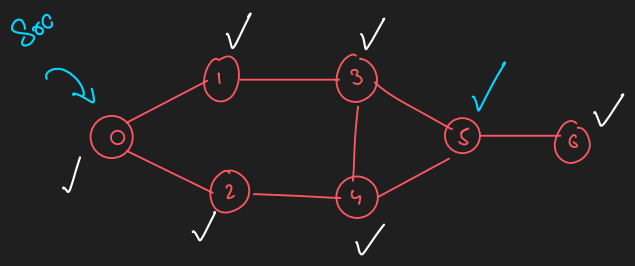


o/p : 0 1 2 3 4 5 6

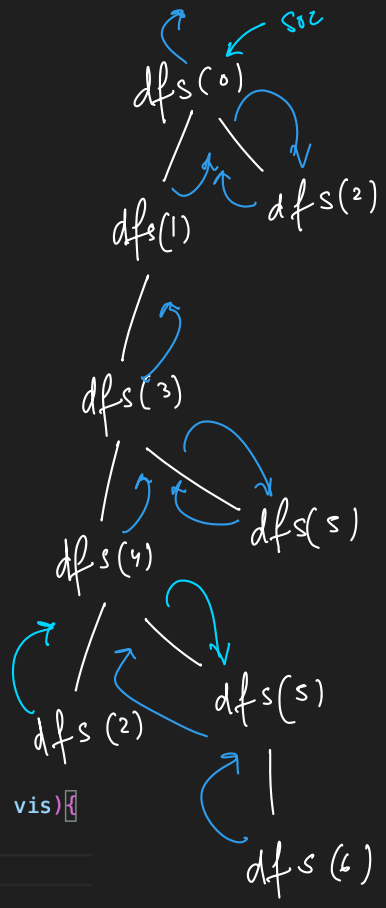


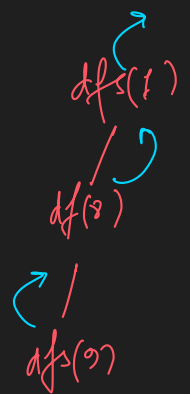

```java
public static void bfs(ArrayList<Edge>[] graph, int src, boolean[] vis){
    Queue<Integer> q = new LinkedList<>();
    q.add(src);

    while(!q.isEmpty()){
        int curr = q.remove();

        if(!vis[curr]){
            vis[curr] = true; // step 1 make visited of curr node true

            // Step 2 : add neighbour
            // neigh : e.dest
            for(int i=0; i<graph[curr].size(); i++){
                Edge e = graph[curr].get(i);
                if(!vis[e.dest]){
                    q.add(e.dest);
                }
            }

            // Step 3 : print
            System.out.print(curr+" ");
        }
    }

    System.out.println();
}
```

DFS



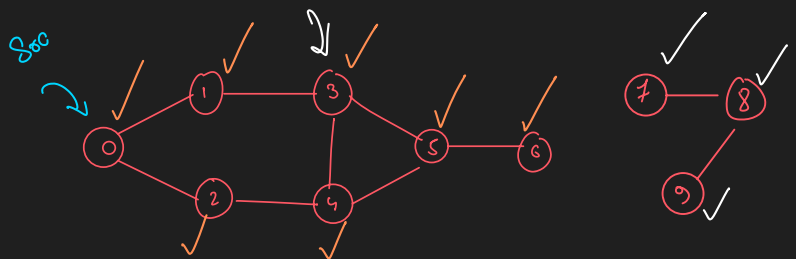
o/p : 0 1 3 4 2 5 6

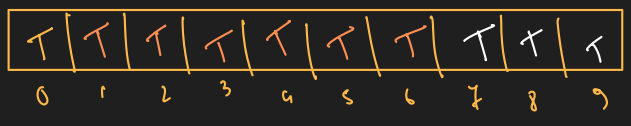





```java
public static void dfs(ArrayList<Edge>[] graph, int curr, boolean[] vis){

    // Step 1 : make visited of curr node true
    vis[curr] =true;

    // Step 2 : print
    System.out.print(curr+" ");

    // Step 3 : visit neighbour
    for(int i=0; i<graph[curr].size(); i++){
        Edge e = graph[curr].get(i);
        if(!vis[e.dest]){
            dfs(graph, e.dest, vis);
        }
    }
}
```

DisConnected Component [Directed Graph]





BFS for dis-connected component

```
bfs ( graph , src ) {

boolean [] vis ;
for ( int i=0 to graph.length ) {
    if ( ! vis [ src ] )
        bfs util ( graph , src , vis )

{
```

bfsUtil (graph, src, vis) {

queue q ;

q.add (src) ;

while (! q.isEmpty()) {

curr = q.remove ()

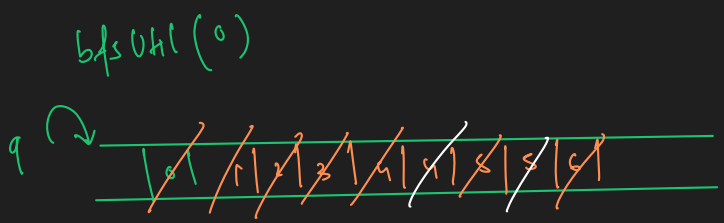
if (! vis [curr]) {

vis [curr] = True ;

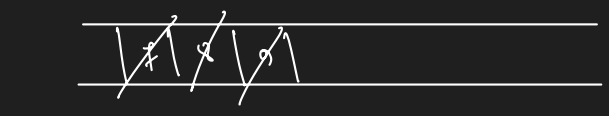
Sout (curr) ;

find unvisited neigh and put it in q.

} }



o/p: 0 1 2 3 4 5 6 7 8 9




```java
public static void bfs(ArrayList<Edge> graph[], int src){
    boolean[] vis = new boolean[graph.length];

    // for(int i=0; i<vis.length; i++){
    //     if(!vis[i]){
    //         bfsUtil(graph, i, vis);
    //     }
    // }
    // System.out.println();

    for(int i=0; i<vis.length; i++){
        int j = (src+i) % vis.length;
        if(!vis[j]){
            bfsUtil(graph, j, vis);
        }
    }

    System.out.println();
}
```

```java
public static void bfsUtil(ArrayList<Edge>[] graph, int src, boolean[] vis){
    Queue<Integer> q = new LinkedList<>();
    q.add(src);

    while(!q.isEmpty()){
        int curr = q.remove();

        if(!vis[curr]){
            vis[curr] = true; // step 1 make visited of curr node true

            // Step 2 : add neighbour
            // neigh : e.dest
            for(int i=0; i<graph[curr].size(); i++){
                Edge e = graph[curr].get(i);
                if(!vis[e.dest]){
                    q.add(e.dest);
                }
            }

            // Step 3 : print
            System.out.print(curr+" ");
        }
    }
}
```

DFS

```java
public static void dfs(ArrayList<Edge>[] graph){
    boolean vis[] = new boolean[graph.length];

    for(int i=0; i<vis.length; i++){
        if(!vis[i]){
            dfsUtil(graph, i, vis);
        }
    }

    System.out.println();
}

public static void dfsUtil(ArrayList<Edge>[] graph, int curr, boolean[] vis){

    // Step 1 : make visited of curr node true
    vis[curr] = true;

    // Step 2 : print
    System.out.print(curr+" ");

    // Step 3 : visit neighbour
    for(int i=0; i<graph[curr].size(); i++){
        Edge e = graph[curr].get(i);
        if(!vis[e.dest]){
            dfsUtil(graph, e.dest, vis);
        }
    }
}
```

```java
Run | Debug
public static void main(String[] args) {
    int V = 10;
    int edgeList[][] = {{0, 1}, {0, 2}, {1, 3}, {2, 4}, {3, 4}, {4, 5}, {3, 5}, {5, 6}, {7, 8}, {8, 9}};

    @SuppressWarnings("unchecked")
    ArrayList<Edge> graph[] = new ArrayList[V];

    createGraph(graph, edgeList);

    bfs(graph, src: 0);

    dfs(graph);
}
```